

STAFF ENGINEER PREPARATION GUIDE — VOLUME 3

How Claude Works

Why AI coding agents understand code, generate designs,
and complete multi-step tasks — and what's really under the hood

It's not "just an LLM": the model, the harness, the agentic loop,
tools, context management, skills, and MCP — in plain language with diagrams

The LLM core • The agent harness • Tool use • Context engineering • Skills & MCP • Myths & limits

Contents

1. **The LLM core** — what a language model actually is, how it's trained, and why that produces coding ability
2. **Not just an LLM** — the agent equation: model + harness + tools + context
3. **The agentic loop** — how one prompt becomes fifty actions
4. **Tool use** — how a text-predictor gets hands
5. **Thinking** — extended reasoning, and why it changes output quality
6. **Context engineering** — the window, caching, compaction, memory, and subagents
7. **Skills** — teachable expertise via progressive disclosure
8. **MCP** — the USB port for AI tools
9. **Why is it good at code and design?** — putting it all together
10. **Myths, limits, and the staff engineer's mental model**

How to use this guide

Same format as Volumes 1 and 2: **Examples** make ideas concrete, **Key ideas** compress each section, **Staff engineer lens** notes connect the concept to decisions you'll make (adopting AI tooling, building on the API, setting team policy). A new block appears in this volume: **Myth vs reality** — common misconceptions worth being able to correct in a design discussion.

Chapter 1 — The LLM Core

What a language model actually does

At the very bottom, a large language model does one thing: given a sequence of **tokens** (word fragments — "refactoring" might be `ref` + `actor` + `ing`), it outputs a probability for every possible next token. Generate one, append it, repeat. That's the whole runtime mechanic.

The interesting question is what it takes to be *good* at that. To predict the next token of `def binary_search(arr, target):` you need to "know" Python syntax. To predict the next line of a half-written function, you need to model the author's *intent*. To continue a code review mid-sentence, you need to have found the bug. **Prediction at this quality forces the model to build internal representations of syntax, semantics, program behavior, and intent** — that's what the billions of parameters learn to encode.



Attention is why code works so well: **the return statement can directly attend to the function signature**, a variable's use can attend to its definition 500 lines earlier — long-range structure is native to the architecture.

Figure 1-1. The core mechanic: tokens in, probability distribution out, one token at a time. Everything else in this guide is built around this loop.

How that ability is trained — three stages

Stage	What happens	What it produces
1. Pretraining	Predict next token across a vast corpus of text and code (public code, docs, books, technical writing)	Raw capability: syntax, APIs, idioms, algorithms, the "shape" of good and bad code — but no notion of being helpful; it just continues text
2. Supervised fine-tuning	Train on curated examples of high-quality assistant behavior: instructions → good responses, bug reports → correct fixes	An assistant: follows instructions, answers rather than merely continues
3. Reinforcement learning (RLHF / Constitutional AI)	Generate responses, score them (human preferences + AI feedback against explicit principles — Anthropic's "Constitutional AI"), reinforce what scores well. For code: reward what <i>actually compiles, runs, and passes tests</i> ; for agents: reward multi-step trajectories that complete real tasks	Judgment: preferring working code over plausible code, honesty over confident nonsense, and knowing when to use a tool vs. answer directly

Stage 3 is the underrated part for coding: code is **verifiable**. Unlike an essay, a program can be executed, tested, and type-checked — so reinforcement learning on code has an objective signal. Modern models are

additionally trained on **agentic trajectories** — full sequences of "read files, edit, run tests, fix failures" — which is why they don't just emit code, they know how to *work* on code.

Example — why it "understands" your codebase without training on it: Claude has never seen the flexone repo. But pretraining exposed it to millions of examples of the *patterns* the repo uses — React components, service classes, migration files, test conventions. When it reads your code (through the context window, at inference time), it maps what it sees onto those learned patterns, the same way a senior engineer joining Flexport is productive in week one: not because they know the codebase, but because they've internalized what codebases are like. "Reading your code" happens in context, at question-time — nothing is learned or stored in the model itself.

Myth vs reality: *"It just memorizes and regurgitates code from GitHub."* Memorization exists at the margins (famous snippets), but the everyday mechanism is generalization: the model writes code for *your* unique combination of function names, business rules, and constraints, which appears nowhere in training data. The proof is mundane: it fixes bugs in code you wrote this morning.

Key idea: An LLM is a next-token predictor whose training (pretraining for capability → fine-tuning for behavior → RL for judgment, with code's verifiability as a strong reward signal) forces it to build deep internal models of language and programs. It learns nothing at inference time — everything it "knows about you" arrives through the context window.

Chapter 2 — Not Just an LLM: The Agent Equation

Here's the direct answer to "is it just an LLM or more things combined?": **the model alone can only read text and emit text.** It cannot open a file, run a test, or search your repo. Everything that makes Claude Code (or any coding agent) feel like a colleague is the model *plus* a system built around it:

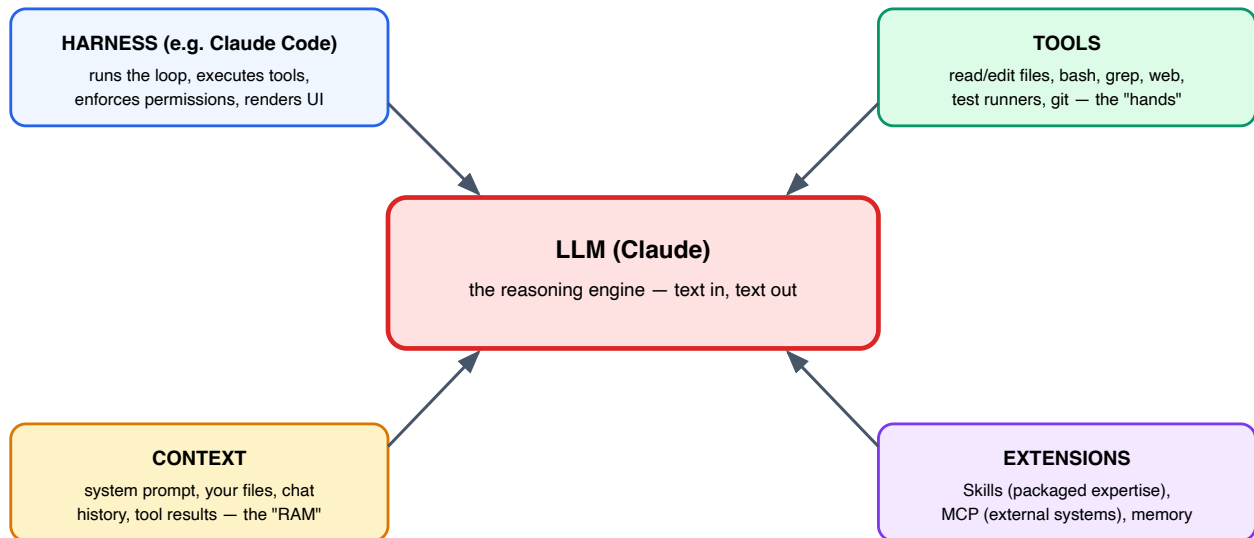


Figure 2-1. The agent equation: $\text{Agent} = \text{Model} + \text{Harness} + \text{Tools} + \text{Context} (+ \text{Skills/MCP})$. Remove any piece and the "colleague" collapses back into a chatbot.

Who does what

- **The model decides; the harness acts.** When Claude "edits a file," the model emits a structured request — "call the Edit tool with this path, this old string, this new string" — and stops. The harness validates it, applies the edit, and feeds the result back. The model never touches your disk.
- **The harness is the security boundary.** Permissions ("ask before running bash"), sandboxing, file-staleness checks (reject an edit if the file changed since it was read), and rendering all live in the harness. This is why the same model behaves differently in Claude Code vs. claude.ai vs. your own API integration — different harnesses, different powers.
- **The context window is the working memory.** The model is **stateless**: every single turn, the entire conversation — system prompt, your messages, every file it read, every tool result — is re-sent and re-read. "Claude remembers what it read" really means "it's still in the context."

Example — one "simple" request, decomposed: You type "fix the failing test in payments." The **harness** wraps that with a system prompt and tool definitions and calls the model. The **model** reasons: "I need to see the failure first" → emits a bash tool call (`npm test`). The harness runs it, appends the output. The model reads the stack trace, emits a Read call for the failing file, then an Edit call with the fix, then bash again to re-run tests. Green. The model writes a summary. **Five model calls, four tool executions, one conversation** — and every step's decision was the LLM, while every step's action was the harness.

Myth vs reality: "*The AI has access to my machine.*" The model has access to **nothing**. It emits structured text; the harness — code you can read, configure, and permission-gate — decides whether to act on it. That distinction is exactly where every real security control lives, and why "which tools does the harness expose, with what approval policy?" is the right security question, not "is the model safe?"

Key idea: "Is it just an LLM?" — No. The LLM is the reasoning engine; a harness supplies the loop, tools supply the hands, the context window supplies working memory, and skills/MCP supply extensibility. The model proposes, the harness disposes.

Staff engineer lens: This decomposition is the adoption framework: evaluating AI tooling means evaluating *harnesses* (what tools, what permission model, what audit trail), not just models. And when someone says "let's build an agent," the model is the easy part — the engineering is the harness: tool design, permissioning, and context management.

Chapter 3 — The Agentic Loop

The difference between a chatbot and an agent is a **while loop**. A chatbot: one question → one answer. An agent: goal → *act*, *observe*, *re-plan*, *act again* — until done. This loop is what turns "write me a function" capability into "migrate this module and make sure the tests pass" capability.

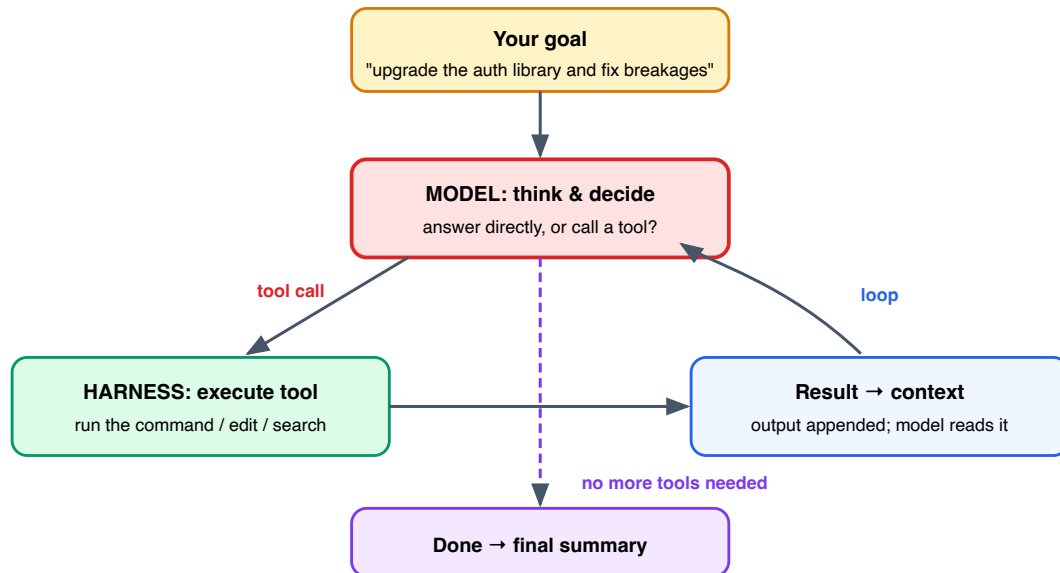


Figure 3-1. The agentic loop: decide → act → observe → decide again. One user request can drive dozens of iterations.

Why the loop is the magic ingredient

It converts prediction into verification. A single-shot model must get code right blind. An agent *runs* the code: if the test fails, the failure output lands in context, and the next iteration reasons about the actual error. Every mistake becomes feedback within the same session. This is why agents drastically outperform single-shot generation on real tasks — not because the model is smarter per call, but because **the loop lets it check its work the way an engineer does**.

It enables exploration. Faced with an unfamiliar repo, the agent does what you'd do: `ls`, `grep` for the symbol, read the failing file, follow imports. Understanding is *gathered*, not assumed.

It recovers. Wrong hypothesis → failed command → visible error → revised plan. A chatbot's wrong answer is final; an agent's wrong answer is an intermediate state.

Example trace — "why is this endpoint slow?":

1. grep for the route → finds the controller
2. Read controller → sees it calls three service methods
3. Read services → spots a query inside a loop (N+1)
4. bash: run the local benchmark → confirms 450ms, dominated by DB
5. Edit: batch the query
6. bash: re-run benchmark → 38ms; run test suite → green
7. Summary with before/after numbers

No step was scripted. The model chose each action *from the previous step's evidence* — that's the loop working as an investigation engine, the same characterize→localize→verify shape as the debugging funnel in Volume 2.

Key idea: Agent = LLM + loop. The loop turns generation into a feedback cycle — act, observe real results, correct — which is why agents can complete tasks no single completion could, and why output quality tracks how *verifiable* the task is (tests to run, compilers to satisfy, pages to screenshot).

Staff engineer lens: Practical corollary: agents do best where feedback is cheap and fast. Investing in your test suite, typed interfaces, and fast local runs doesn't just help humans — it's literally the sensory system an AI agent uses. "Make the repo agent-friendly" ≈ "make feedback loops fast," which you wanted anyway.

Chapter 4 — Tool Use: How a Text-Predictor Gets Hands

Tool use is the bridge between "emits text" and "does things." The mechanism is elegantly simple: tools are **described to the model in the prompt** (name, purpose, JSON schema of parameters), and the model is trained to emit a special structured block when it wants one called.

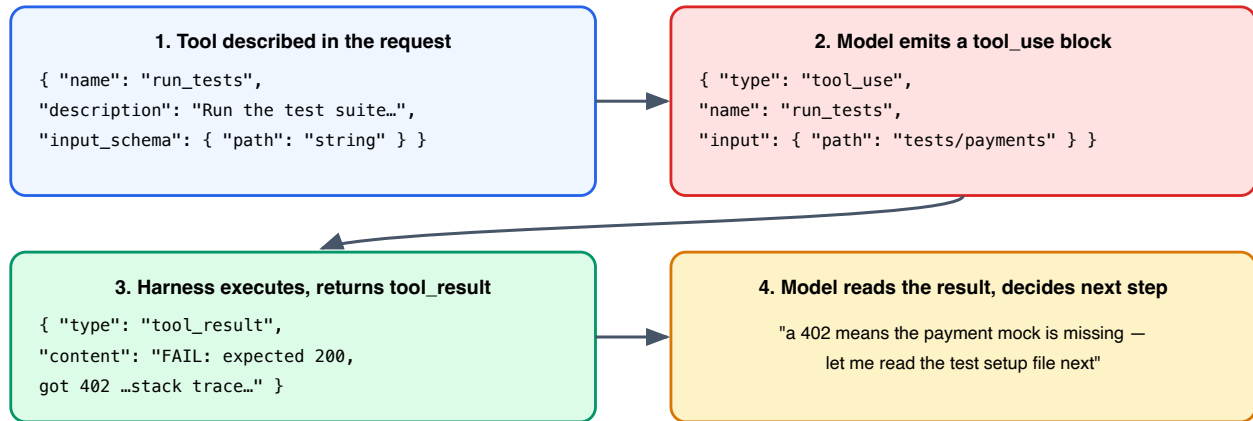


Figure 4-1. The tool-use wire protocol: description in, structured call out, result back in. Steps 2–4 repeat as the agentic loop.

The tool surface is a design decision

Which tools an agent gets — and their granularity — shapes both capability and safety:

- **Broad tools (bash)**: maximum leverage, minimum oversight — the harness sees only an opaque command string.
- **Dedicated tools (Read/Edit/Grep)**: typed parameters the harness can inspect, gate, and even enforce invariants on (e.g., Claude Code's Edit tool rejects a write if the file changed since it was last read — a staleness check bash could never provide).
- **Approval-gated tools**: anything irreversible or externally visible (push, send message, deploy) can require human confirmation *at the harness level*, regardless of what the model wants.

Two refinements matter at scale: **parallel tool calls** (the model batches independent reads/greps into one turn — meaningfully faster) and **tool search** (with hundreds of available tools, definitions are loaded on demand rather than stuffing every schema into every prompt).

Example — why descriptions matter more than code: A team wired up a `query_datadog` tool, and the agent never used it. The tool's description said "Queries Datadog." Rewritten to "Call this when you need production metrics, error rates, or latency data — e.g. to check if a bug affects real traffic," usage became reliable. **The description is the interface**: the model chooses tools by reading them, exactly as a new engineer chooses functions by their docstrings. Writing tool descriptions is prompt engineering, not documentation.

Key idea: Tools = schemas in the prompt + structured emissions + harness execution. The model never executes anything. Tool *choice* is driven by descriptions; tool *safety* is enforced by the harness; tool *granularity* is the lever trading leverage against control.

Staff engineer lens: If you build agents on the API, budget most of your design time here: which actions deserve dedicated tools vs bash, which need approval gates, what invariants the harness enforces. "Start with bash for breadth; promote an action to a dedicated tool when you need to gate, render, audit, or parallelize it" is the working heuristic.

Chapter 5 — Thinking: Reasoning Before Answering

Modern Claude models can **think before responding** — generating internal reasoning tokens that are separate from the final answer. This isn't a UI gimmick; it changes what the model can do.

Why generating "thinking" tokens helps

Recall the core mechanic: one token at a time, each conditioned on everything before it. A model answering instantly must compress all its reasoning into the forward pass that produces the first answer token. Thinking tokens give it **scratch space**: it can lay out the problem, consider approaches, catch its own contradiction, and revise — and then the final answer is conditioned on all of that written-down reasoning. It's the same reason you work a hard problem on paper instead of in your head: intermediate results externalized are intermediate results you can build on.

- **Adaptive thinking:** the model decides *when and how much* to think — a rename gets near-zero thinking; a gnarly concurrency bug gets a lot. It also thinks *between tool calls*: after reading a stack trace, before choosing the next action.
- **Effort levels:** a dial (low → max) trading depth for latency/cost. High effort on an agentic task doesn't just mean longer answers — it means more verification behavior: re-reading, double-checking, running the tests again.

Example: Ask for the write-skew explanation from Volume 1 and the answer comes back with barely any thinking — it's retrieval-shaped. Ask "*here are 400 lines of a race condition we see once a week in prod — find it*" and the model may spend thousands of thinking tokens tracing interleavings before answering. Same model, self-calibrated depth — this is why "AI is instant" and "AI takes minutes" are both true.

Key idea: Thinking = the model using its own output as scratch space, making multi-step reasoning conditionable. Adaptive thinking calibrates the depth per task and interleaves reasoning between tool calls — one of the main reasons agent step-choices are good.

Chapter 6 — Context Engineering: The Memory Problem

The context window (up to ~1M tokens — several large books) is the agent's entire working memory, and it is the binding constraint of agent design. Three hard facts drive everything:

1. **Stateless:** every turn re-sends and re-processes the whole conversation.
2. **Finite:** long agentic sessions — dozens of file reads and test outputs — genuinely fill it.
3. **Attention degrades:** a window stuffed with stale tool output dilutes focus on what matters now (the "needle in a haystack" cost is real even before the hard limit).

The four tools of context management

Four strategies, four different problems

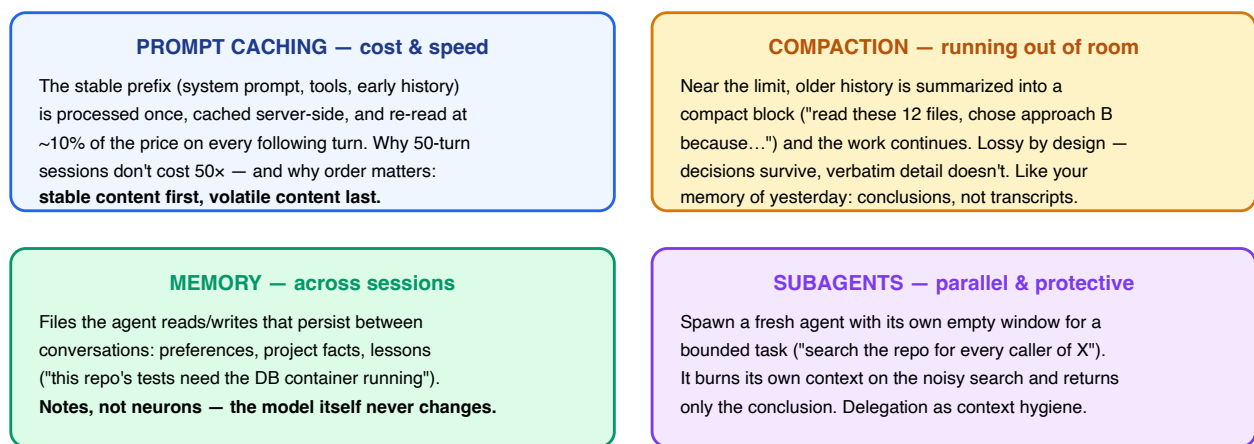


Figure 6-1. Caching solves cost, compaction solves overflow, memory solves cross-session persistence, subagents solve pollution. Real agents use all four.

Example — why your CLAUDE.md and memory files work: When Claude "remembers" that you prefer PRs cut from fresh main, no model was retrained. A memory file was written during an earlier session; at the start of this one, the harness loaded it into context. Editing that file *is* editing the agent's long-term memory — which is also why it's auditable and correctable in a way "learned" behavior wouldn't be.

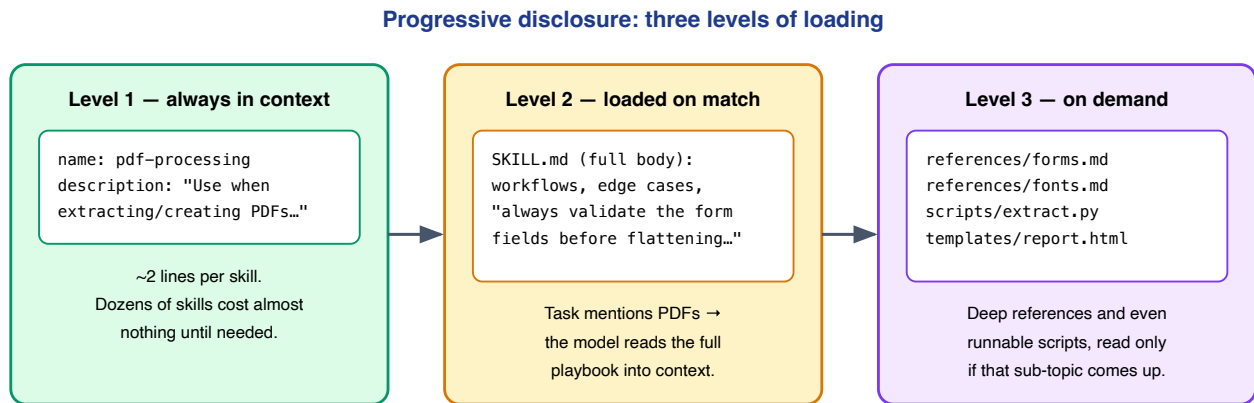
Myth vs reality: "It learns from our codebase over time." The model's weights are frozen at training. All apparent learning is context: memory files, CLAUDE.md conventions, and what's been read this session. Restart with empty context and it's a blank slate. Implication: anything you want the agent to reliably know belongs in **written, versioned artifacts** — which is the same discipline that helps human onboarding.

Key idea: Context is the scarce resource of agentic AI. The craft — "context engineering" — is keeping the window relevant: cache the stable, compact the old, persist the durable to files, and delegate the noisy to subagents.

Staff engineer lens: Two takeaways. (1) Repos with good written context — CLAUDE.md, READMEs, ADRs, descriptive tests — get visibly better agent output; documentation quality is now an automation multiplier, an argument that finally gets docs prioritized. (2) If you build on the API, cache-friendliness is an architecture concern: a timestamp interpolated into the system prompt silently invalidates the cache every request — the classic "silent invalidator" bug.

Chapter 7 — Skills: Teachable Expertise

You asked specifically how skills work — here is the whole mechanism, and it's refreshingly low-tech: **a skill is a folder of instructions the model reads when relevant**. No fine-tuning, no plugins in the traditional sense, no code injection into the model. Structured documentation, loaded just-in-time.



Same pattern as good docs: a table of contents you skim, chapters you open when relevant, appendices you consult rarely. **Expertise scales without context cost.**

Figure 7-1. Progressive disclosure: descriptions always visible, bodies loaded on relevance, references on demand.

Why this design wins

- **It rides the model's strongest skill — reading.** No new mechanism needed: the model already follows instructions superbly (that's what fine-tuning optimized), so packaged instructions are packaged capability.
- **Cheap specialization.** Fine-tuning a model on your team's deploy process costs GPUs and expertise, and it fossilizes. A skill is a markdown file: written in an afternoon, versioned in git, code-reviewed, updated when the process changes.
- **Composable and portable.** A task touching Excel and your internal billing conventions loads both skills. The same skill folder works across Claude Code, the API, and Managed Agents.
- **Skills can carry code.** A skill may bundle scripts the model *executes* rather than re-derives — the PDF skill ships a battle-tested extraction script instead of hoping the model writes one correctly each time. Instructions + tools in one package.

Example — a team skill you could write this week: A `flexport-migrations` skill: description "Use when creating or reviewing DB migrations." Body: the `expand-migrate-contract` rules from Volume 2, the team's naming conventions, the `"always CREATE INDEX CONCURRENTLY"` rule, and a checklist. Now every engineer's agent session enforces the team's hard-won migration discipline automatically — **a code review comment you write once and never again**. This is the same mechanism by which this very session loaded an `"n8n workflow"` or `"Claude API"` skill: relevant folder, read on demand.

Myth vs reality: "*Skills are plugins that extend the model.*" Nothing executes inside the model, and the model doesn't change. A skill is *context on demand* — the same tokens-in-tokens-out machine reading a really good runbook at the right moment. That's also why skill quality $\approx$ writing quality: a vague skill is exactly as useless as a vague wiki page.

Key idea: Skills = folders of markdown (+ optional scripts) with a two-line description always in context and everything else progressively disclosed. They turn institutional knowledge into agent capability at the cost of writing it down — no training, fully versionable, code-reviewable.

Staff engineer lens: Skills are the highest-leverage, lowest-risk AI investment a team can make: identify what you repeatedly explain in code review (migration rules, incident runbooks, API conventions) and package it. It's documentation with an enforcement mechanism — and unlike a wiki, it's *read every time it's relevant*.

Chapter 8 — MCP: The USB Port for AI Tools

Tools (Ch. 4) explain how the model calls a function. The **Model Context Protocol** answers the scaling question: how do a hundred different AI applications connect to a thousand different systems — Slack, Jira, Datadog, your internal finance service — without writing 100×1000 custom integrations?

MCP is an open standard (originated by Anthropic, now industry-wide) that fixes the *shape* of the connection: an **MCP server** wraps any system and exposes its capabilities as self-describing tools (name + description + JSON schema — exactly the format of Ch. 4); an **MCP client** (Claude Code, claude.ai, your own app) connects to any server and discovers its tools at runtime.

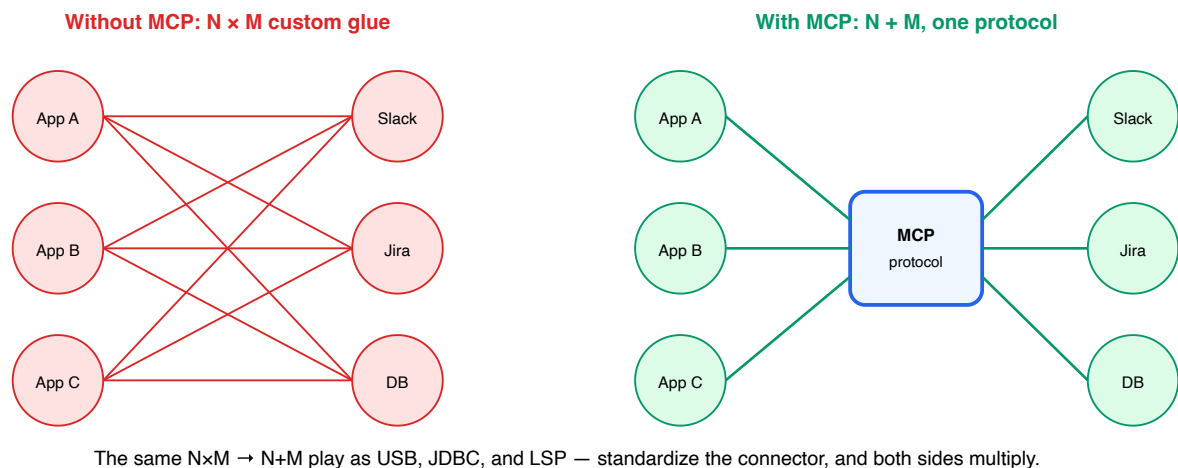


Figure 8-1. MCP turns the integration matrix into a hub: any MCP client can use any MCP server, discovered at runtime.

Once connected, MCP tools are indistinguishable from built-in tools from the model's perspective — same describe→call→result cycle. Because tool definitions arrive at *runtime*, the agent gains capabilities without anyone retraining or even redeploying the model. Auth stays at the connection layer (OAuth to the server), so the model never holds credentials.

Example — this session, literally: The Claude session that produced this document is connected to MCP servers for Slack, Datadog, Sentry, Buildkite, and more. None of those integrations are "in" Claude. Each is a server exposing tools like `search_datadog_logs`; the harness listed them, the model can call them. Your finance platform could expose one — `get_invoice`, `reconcile_period` — and every MCP-speaking agent in the company could operate it, with the security review happening once, on the server.

Myth vs reality: "Connecting Claude to X means giving the model access to X." The MCP *client* (harness) holds the connection and credentials; the model emits call-requests that the harness forwards subject to permissions. Scope, auditing, and read-only enforcement all live server- and harness-side. The right review question for an MCP integration is the same as for any service account: what can this credential do, and who approves the writes?

Key idea: MCP standardizes how agents discover and call external capabilities — $N + M$ instead of $N \times M$. Skills package *knowledge*; MCP connects *systems*; both extend the agent without touching the model.

Chapter 9 — Why Is It Actually Good at Code and Design?

With all the pieces on the table, the answer to your original question is a stack of five reinforcing layers:

Layer	Contribution to coding/design ability
1. Training distribution	Enormous exposure to code, reviews, docs, design systems, and UI code — the priors for "what good looks like" in both engineering and visual design
2. Verifiable-domain RL	Code compiles, tests pass, types check: reinforcement learning had an objective reward, plus explicit training on multi-step agentic trajectories — so the model learned to <i>work</i> , not just to emit
3. Attention over long context	Holding your actual files in the window while generating — usage sites attend to definitions, new code attends to your conventions; this is "codebase awareness"
4. Thinking	Scratch-space reasoning before answering and between tool calls — planning, tracing, self-correction
5. The agentic loop + tools	Run it, see the failure, fix it, re-run — converts probabilistic generation into verified results; for design, screenshot the page and iterate on what it <i>looks like</i>

Why design too, not just code?

Frontend design sits at a lucky intersection: it's **expressed as code** (HTML/CSS/SVG — fully in-distribution), it's **drenched in written principles** (decades of typography, spacing, color-theory, and accessibility writing to learn from), and it's **verifiable-ish** — an agent can render its output, screenshot it, and critique the screenshot with its own vision capability, then iterate. The diagrams in these three PDFs were produced exactly that way: SVG generated as code, rendered, inspected, adjusted. What the model lacks is *taste grounded in your users* — it produces competent, principle-following design and needs human direction for distinctive design (which is why "propose 4 directions, let me pick" beats "make it pretty").

Example — the layers working together on a real task: "Add currency-rounding validation to invoice submission." Layer 3 reads your validator conventions from context; layer 1 already knows banker's-rounding pitfalls; layer 4 thinks through which of your three validation layers should own the rule; layer 5 writes the change, runs your test suite, sees an unrelated-looking failure, investigates (it wasn't unrelated — a fixture assumed the old behavior), fixes the fixture, re-runs green. No single layer could do this: remove the loop and the fixture bug ships; remove context and the code ignores your conventions; remove training and there's nothing to reason *with*.

Key idea: Coding ability = trained priors × in-context codebase awareness × reasoning × verified iteration. It's a systems answer, not a model answer — which is precisely why the question "is it just an LLM?" has a genuinely interesting "no."

Chapter 10 — Myths, Limits, and the Mental Model to Keep

Limits that follow directly from the mechanics

Limit	Mechanical cause	Working mitigation
Hallucination (confident wrong API, invented function)	The generator produces <i>plausible</i> tokens; plausibility $\neq$ truth, and there's no built-in fact-database lookup	Verifiability: run/compile/test everything; ground claims in tool results ("show me the failing test output")
Knowledge cutoff	Weights frozen at training; yesterday's library release isn't in them	Tools: web search/fetch, reading the installed package's actual source — retrieval beats recall
Context loss in marathon sessions	Compaction is lossy; late-session references to early verbatim detail degrade	Durable decisions → files (plans, ADRs, memory), not chat history; fresh sessions for fresh workstreams
Over-agreeableness	RLHF optimizes for human approval, which correlates with agreeing	Ask for the case <i>against</i> your idea explicitly; frame reviews adversarially ("try to refute this")
Non-determinism	Sampling from a distribution; same prompt $\neq$ same output	Treat outputs like a colleague's draft — review gates, tests, CI — not like a compiler's deterministic output
Prompt injection	The model reads instructions and data through the same channel (tokens); malicious text in a fetched page can <i>look like</i> instructions	Harness-level defense: least-privilege tools, approval gates on writes/sends, treating external content as untrusted

The mental model that survives contact with reality

Not "magic intern," not "fancy autocomplete." The most productive frame: **an extremely well-read, fast, tireless engineer with no memory of your company beyond what's written down, whose work you verify the same way you verify anyone's — through tests, review, and evidence.** Every practice that makes that engineer effective is a practice you already value: written conventions, fast test feedback, clear task specifications, honest review.

Staff engineer lens — the interview-ready summary: If asked "how do these AI coding tools actually work?", the 60-second answer: "A *transformer LLM trained on next-token prediction over code — with RL against verifiable rewards like passing tests — sits inside a harness that runs an agentic loop: the model emits structured tool calls, the harness executes them (file edits, bash, search) and feeds results back, so the model iterates against real feedback instead of generating blind. Working memory is a context window managed by caching, compaction, file-based memory, and subagents; capabilities extend without retraining via skills — instruction folders loaded on demand — and MCP, a standard protocol for connecting external systems as tools. The model decides, the harness acts — so safety, permissions, and audit all live in the harness.*" Every clause maps to a chapter of this volume.

The volume in three sentences

1. The core is a next-token predictor whose training on vast code (with verifiable rewards) produced deep priors — but the model alone only reads and emits text, learns nothing at runtime, and touches nothing directly. **2.** Agents emerge from the harness: a loop in which the model emits structured tool calls, real results come back as context, and generation becomes verified iteration — with thinking supplying scratch-space reasoning at each decision point. **3.** Everything "extra" is context engineering — caching, compaction, memory files, subagents, skills (knowledge on demand), MCP (systems on demand) — which means the levers you control are written artifacts and harness design, not the model.